UNIVERSIDAD SAN CARLOS DE GUATEMALA
CENTRO UNIVERSITARIO DE OCCIDENTE
DIVISIÓN DE CIENCIAS DE LA INGENIERÍA
ORGANIZACIÓN DE LENGUAJES Y COMPILADORES 1

Manual Técnico - Práctica 2

Mauricio Joel Gómez Barrios - 202031478

11 de abril de 2026

Índice.

Introducción **3**
Organización del Proyecto - Backend **4**
Reglas léxicas **5**
Reglas Sintácticas **6**
Procedimiento para construir gramáticas en Wison 7
Analizador Léxico de Wison (wison.jison – sección del lex) **7**
● Palabras Reservadas 7
● Identificadores Especiales 7
● Expresiones Literales y Conjuntos 7
● Manejo de Errores Léxicos 8
Analizador Sintáctico de Wison (wison.jison – sección gramatica) **8**
Estructura del AST producido 8
Nodos de expresiones regulares 8
Gestor de Gramática (GestorGramatica.js) **9**
Tabla de Símbolos (generarTablaSimbolos) 9
Cálculo de Conjuntos First (calcularFirsts) 9
Cálculo de Conjuntos Follow (calcularFollows) 9
Generación de la Tabla Predictiva – TAS (generarTablaParser) 9
Análisis LL(1) con Pila (analizarCadenaConPila) 9
Lexer Dinámico (Lexer.js) **10**
Método generarTokens 10
Método verificarAST 10
Organización del Proyecto - Frontend **11**
Servidor HTTP (server.js) **11**
Endpoint: POST /api/compilar 11
Interfaz de Usuario – Angular **11**
api.ts – Servicio de Comunicación 11
app.ts – Componente Principal 11
app.html – Plantilla 12
app.css – Estilos 12
Diagrama de Clases **13**

Introducción

La aplicación es un compilador web desarrollado cuyo propósito es permitir al usuario definir gramáticas libres de contexto usando el lenguaje wison, verificar que dichas gramáticas sean válidas para un analizador LL(1), y evaluar cadenas de entrada generando el árbol de derivación correspondiente.

La solución se divide en dos grandes capas: un backend en Node.js que expone una API REST y realiza todo el procesamiento como análisis léxico y sintáctico de wison, cálculo de first y follow, construcción de la tabla predictiva y análisis LL(1) y un frontend en angular que proporciona la interfaz gráfica donde el usuario escribe la gramática, ingresa cadenas de prueba y visualiza los resultados.

Organización del Proyecto - Backend.

Organización del proyecto cliente y servidor:

Archivo	Responsabilidad
wison.jison	Gramática fuente del lenguaje Wison; Jison la compila al parser.
analizadorWison .js	Parser generado por Jison. Recibe código Wison y retorna un AST.
GestorGramatic a.js	Tabla de símbolos, first, follow, TAS y análisis LL(1) con pila.
Lexer.js	Lexer dinámico: convierte texto de prueba en tokens usando los terminales del AST.
server.js	API REST. Endpoint POST /api y compilar maneja todo el backend.
app.ts / app.html / app.css	Componente principal Angular: formulario, envío al API y visualización de resultados.
api.ts	Servicio Angular (HttpClient) que encapsula la llamada al backend.

Flujo de ejecución:

- El usuario escribe una gramática Wison y una cadena de prueba en el frontend.
- El servicio api.ts realiza un POST a /api/compilar con ambos textos.
- server.js invoca al parser Jison para obtener el AST.
- GestorGramatica construye la tabla de símbolos, calcula First y Follow, genera la TAS y analiza la cadena con la pila LL(1).
- El servidor responde con First, Follow, TAS, árbol de derivación y lista de errores.
- El frontend genera las tablas y el árbol de derivación de forma visual.

Reglas léxicas

Token	Valor	Descripción
RW_WISON_INICIO	Wison¿	Inicio de la estructura Wison
RW_WISON_FIN	?Wison	Cierre de la estructura Wison
RW_LEX_INICIO	Lex {: (con espacios)	Inicio del bloque léxico
RW_LEX_FIN	:}	Cierre del bloque léxico
RW_SYNTAX_INICIO	Syntax {{:	Inicio del bloque sintáctico
RW_SYNTAX_FIN	:}}	Cierre del bloque sintáctico
RW_TERMINAL	Terminal	Declaración de un terminal
RW_NO_TERMINAL	No_Terminal	Declaración de un no terminal
RW_INITIAL_SIM	Initial_Sim	Símbolo inicial de la gramática
FLECHA_LEX	<-	Asignación en la parte léxica
FLECHA_SYNTAX	<=	Asignación en producciones
PUNTOCOMA	;	Terminador de instrucción
OR	|	Alternativa en producciones
OPE_KLEENE	*	Estrella de Kleene (0 o más)
OPE_POSITIVA	+	Cerradura positiva (1 o más)
OPE_OPCIONAL	?	Opcional (0 o 1 vez)
PAR_ABRIR / PAR_CERRAR	( )	Agrupación/concatenación en regex
ID_TERMINAL	$_[a-zA-Z0-9_]+	Nombre de un terminal
ID_NOTERMINAL	%_[a-zA-Z0-9_]+	Nombre de un no terminal
CADENA	'<texto>'	Literal de caracteres
CONJUNTO_LETRAS	[aA-zZ]	Rango de letras (alfabeto completo)
CONJUNTO_NUMEROS	[0-9]	Rango de dígitos
(ignorado)	\s+	Espacios, tabs y saltos de línea
(ignorado)	#...	Comentario de línea
(ignorado)	/** ... */	Comentario de bloque

Reglas Sintácticas

La gramática completa aceptada por el parser Jison es la siguiente:

No Terminal	Producción	Descripción
inicio	RW_WISON_INICIO bloque_lexico bloque_syntax RW_WISON_FIN EOF	Raíz
bloque_lexico	RW_LEX_INICIO lista_terminales RW_LEX_FIN	Bloque Lex
lista_terminales	lista_terminales instruccion_terminal	Lista (recursiva)
lista_terminales	instruccion_terminal	Lista (base)
instruccion_terminal	RW_TERMINAL ID_TERMINAL FLECHA_LEX expresion_lexica PUNTOCOMA	Declaración terminal
expresion_lexica	expresion_lexica OPE_KLEENE	Kleene (*)
expresion_lexica	expresion_lexica OPE_POSITIVA	Positiva (+)
expresion_lexica	expresion_lexica OPE_OPCIONAL	Opcional (?)
expresion_lexica	PAR_ABRIR lista_concat PAR_CERRAR	Concatenación
expresion_lexica	CADENA	Literal string
expresion_lexica	CONJUNTO_LETRAS	Rango letras
expresion_lexica	CONJUNTO_NUMEROS	Rango dígitos
expresion_lexica	ID_TERMINAL	Referencia terminal
lista_concat	lista_concat expresion_lexica	Concatenación recursiva
lista_concat	expresion_lexica	Concatenación base
bloque_syntax	RW_SYNTAX_INICIO declaraciones_syntax RW_SYNTAX_FIN	Bloque Syntax
declaraciones_syntax	declaraciones_syntax declaracion_syntax	Declaraciones recursivas
declaraciones_syntax	declaracion_syntax	Declaración base
declaracion_syntax	decl_no_terminal	Tipo no terminal
declaracion_syntax	decl_initial_sim	Tipo símbolo inicial
declaracion_syntax	produccion	Tipo producción
decl_no_terminal	RW_NO_TERMINAL ID_NOTERMINAL PUNTOCOMA	Declarar NT
decl_initial_sim	RW_INITIAL_SIM ID_NOTERMINAL PUNTOCOMA	Símbolo inicial
produccion	ID_NOTERMINAL FLECHA_SYNTAX lista_alternativas PUNTOCOMA	Regla de producción
lista_alternativas	lista_alternativas OR cuerpo_produccion	Alternativas recursivas
lista_alternativas	cuerpo_produccion	Alternativa base
cuerpo_produccion	cuerpo_produccion simbolo_produccion	Cuerpo recursivo
cuerpo_produccion	simbolo_produccion	Cuerpo base
simbolo_produccion	ID_TERMINAL	Terminal en cuerpo
simbolo_produccion	ID_NOTERMINAL	No terminal en cuerpo

Procedimiento para construir gramáticas en Wison
Para definir una gramática válida en el lenguaje Wison se deben seguir los pasos siguientes:

1. **Iniciar la estructura con la palabra reservada Wison¿**
   Toda definición de gramática debe estar encapsulada entre Wison¿ y ?Wison.

2. **Definir la parte léxica dentro del bloque Lex {: ... :}**
   Cada terminal se declara con la sintaxis:
   Terminal $_NOMBRE <- EXPRESION_REGULAR ;
   Las expresiones regulares soportadas son: cadenas entre comillas simples ('abc'), [aA-zZ] para letras, [0-9] para dígitos, los operadores *, + y ?, y la concatenación usando paréntesis. Se pueden referenciar terminales ya definidos.

3. **Definir la parte sintáctica dentro del bloque Syntax {{: ... :}}**
   Dentro de este bloque se siguen tres pasos en orden:
   - Declarar todos los no terminales antes de usarlos: No_Terminal %_Nombre ;
   - Especificar el símbolo inicial: Initial_Sim %_Nombre ;
   - Definir las producciones usando: %_Cabeza <= cuerpo1 | cuerpo2 ;

4. **Cerrar con ?Wison.**
   Restricciones de la gramática para que sea LL(1):
   - No debe existir recursividad por la izquierda directa (A -> A ...).
   - No debe haber ambigüedad: dos alternativas del mismo no terminal no pueden comenzar con el mismo símbolo.
   - Todos los no terminales deben declararse antes de referenciarse en las producciones.
   - El símbolo inicial es obligatorio

Analizador Léxico de Wison (wison.jison – sección del lex)
En wison.jison en la sección %lex el analizador léxico. Jison la compila y genera el código JavaScript equivalente dentro de analizadorWison.js. Sus características son:

- **Palabras Reservadas**
  Las palabras clave de Wison son case-sensitive. El lexer las reconoce como patrones literales con mayor prioridad que los identificadores: Wison¿, ?Wison, Lex {:, Syntax {{:, :}}, :}, Terminal, No_Terminal, Initial_Sim.

- **Identificadores Especiales**
  Los nombres de terminales deben iniciar con $_ seguido de caracteres alfanuméricos o guión bajo ($_ [a-zA-Z0-9_]+), y los nombres de no terminales con %_ (%_[a-zA-Z0-9_]+). Esta distinción de prefijo evita ambigüedades en la gramática.

- **Expresiones Literales y Conjuntos**
  Las cadenas se capturan como la secuencia '[^']+' cualquier texto entre comillas simples. Los conjuntos especiales [aA-zZ] y [0-9] son tokens propios del lenguaje, no expresiones regulares genéricas; el lexer los reconoce como tokens CONJUNTO_LETRAS y CONJUNTO_NUMEROS respectivamente.

- **Manejo de Errores Léxicos**
  La regla final . actúa como capturador: ante cualquier carácter no reconocido lanza un Error con el mensaje 'Error léxico: carácter inesperado ... en línea N', deteniendo el análisis inmediatamente..

Analizador Sintáctico de Wison (wison.jison – sección gramatica)
La sección de gramática del archivo wison.jison define las producciones del lenguaje Wison. Jison genera un parser LALR(1) a partir de ellas. Cada acción semántica (bloque { ... }) construye el AST que luego consume GestorGramatica.

Estructura del AST producido
El parser retorna un objeto con la siguiente forma:

```
{
  tipo: "GRAMATICA_WISON",
  lexico: [                    // Arreglo de terminales
    { id: "$_NOMBRE", expresion: <nodo_regex> }, ...
  ],
  sintactico: [                // Arreglo de instrucciones
    { tipo: "NO_TERMINAL",  id: "%_X" },
    { tipo: "INITIAL_SIM",  id: "%_S" },
    { tipo: "PRODUCCION",   cabeza: "%_S", alternativas: [[...]] }
  ]
}
```

Nodos de expresiones regulares
Cada expresión regular del bloque Lex se representa como un árbol de nodos:

tipo	Campos adicionales	Descripción
CADENA	valor: "'abc'"	Literal de texto entre comillas simples
LETRAS	(ninguno)	Rango [aA-zZ]
NUMEROS	(ninguno)	Rango [0-9]
KLEENE	expr: <nodo>	Operador * sobre la expresión hija
POSITIVA	expr: <nodo>	Operador + sobre la expresión hija
OPCIONAL	expr: <nodo>	Operador ? sobre la expresión hija
CONCAT	exprs: [<nodo>, ...]	Concatenación de expresiones dentro de ()
REF_TERMINAL	id: "$_X"	Referencia a otro terminal ya declarado

Gestor de Gramática (GestorGramatica.js)
Esta clase es el núcleo del backend. Recibe el AST producido por el parser Jison y realiza todos los cálculos necesarios para operar como un compilador LL(1). Su constructor hace automáticamente la construcción de la tabla de símbolos y las verificaciones de recursividad y factorización.

Tabla de Símbolos (generarTablaSimbolos)
Recorre el AST en dos pasadas:

- Primera pasada (léxico): registra cada terminal (id → nodo de expresión regular) en un map. Detecta duplicados.
- Primera pasada (sintáctico – declaraciones): registra no terminales en un set y el símbolo inicial. Detecta si se volvió a declarar y la ausencia de símbolo inicial.
- Segunda pasada (sintáctico – producciones): valida que cada símbolo referenciado tanto terminales como no terminales haya sido declarado previamente y registra las producciones en el mapa de producciones.

Los errores semánticos detectados se acumulan en el arreglo de errores.

Cálculo de Conjuntos First (calcularFirsts)
Implementa un algoritmo estándar. Inicializa el conjunto First de cada no terminal como vacío y repite hasta que no haya cambios:

- Si la alternativa es vacía (epsilon), agrega ε al First de la cabeza.
- Si la alternativa comienza con un Terminal, agrega ese terminal al First de la cabeza.
- Si la alternativa comienza con un No Terminal, hereda su conjunto First (sin ε si no lo contiene).

El algoritmo garantiza convergencia porque los conjuntos solo crecen.

Cálculo de Conjuntos Follow (calcularFollows)
Implementa el algoritmo iterativo estándar:

- Agrega $ al Follow del símbolo inicial.
- Para cada símbolo B en la parte derecha de una producción $A \rightarrow \alpha B \beta$, agrega $First(\beta) \setminus \{\varepsilon\}$ al Follow(B).
- Si β puede derivar ε (o B está al final de la producción), agrega Follow(A) al Follow(B).

Generación de la Tabla Predictiva – TAS (generarTablaParser)
Genera la Tabla de Análisis Sintáctico M[NT, T]. Para cada producción $A \rightarrow \alpha$:

- Calcula First(α). Para cada terminal a en $First(\alpha) \setminus \{\varepsilon\}$, asigna $M[A, a] = \alpha$.
- Si $\varepsilon \in First(\alpha)$, para cada terminal b en Follow(A) asigna $M[A, b] = \alpha$.
- Si se detecta una celda ya tiene asignada una producción, registra un error indicando que la gramática no es LL(1) puro.

El token de fin de cadena se trata internamente como '$_FIN' para mantener consistencia con los identificadores de terminales de Wison.

Análisis LL(1) con Pila (analizarCadenaConPila)
Implementa el algoritmo clásico de análisis descendente con pila predictiva:

- Inicializa la pila con $_FIN en el fondo y el símbolo inicial en el tope.
- Si el tope de la pila es un Terminal y coincide con el token actual, encuentra a un igual (pop y avanza el puntero de entrada).
- Si el tope es un No Terminal, consulta la TAS M[NT, token]. Si existe la producción, hace pop e introduce los símbolos del cuerpo en orden inverso.

- Si en cualquier paso no hay coincidencia ni entrada en la TAS, retorna { exito: false }.
- Simultáneamente construye el árbol de derivación como un objeto JSON de nodos con campos etiqueta e hijos, que el frontend renderiza visualmente.

Lexer Dinámico (Lexer.js)
Este lexer a diferencia del wison es dinámico: recibe el mapa de terminales producido por GestorGramatica y es capaz de hacer tokens cualquier texto de entrada usando las expresiones regulares definidas por el usuario en Wison.

Método generarTokens
Recorre la cadena de entrada carácter a carácter:
- Omite espacios en blanco, tabulaciones y saltos de línea.
- Para cada posición, prueba todos los terminales registrados llamando a verificarAST.
- Aplica lo de máximos iguales: elige siempre el token con el lexema más largo.
- Si ningún terminal tiene igual, lanza un Error léxico con la posición exacta.
- Al finalizar, agrega automáticamente el token { id: '$_FIN', lexema: 'EOF' }.

Método verificarAST
Interpreta el árbol de expresión regular recursivamente. Cada tipo de nodo tiene su propia lógica:

Tipo de nodo	Comportamiento
CADENA	Compara la subcadena en la posición actual con el valor literal sin comillas simples. Retorna la longitud si hay igual, -1 si no.
LETRAS	Verifica si el carácter actual es una letra (a-z o A-Z). Retorna 1 o -1.
NUMEROS	Verifica si el carácter actual es un dígito (0-9). Retorna 1 o -1.
CONCAT	Aplica verificarAST a cada subexpresión en secuencia, acumulando el avance. Si cualquiera falla, retorna -1.
POSITIVA	Requiere al menos una coincidencia, luego aplica Kleene. Retorna -1 si la primera falla.
KLEENE	Aplica la expresión 0 o más veces mientras haya coincidencia. Siempre retorna un valor >= 0.
OPCIONAL	Intenta la coincidencia; si falla retorna 0 sin error.
REF_TERMINAL	Busca el terminal referenciado en el mapa y llama a verificarAST recursivamente con su expresión.

Organización del Proyecto - Frontend.

Servidor HTTP (server.js)

El servidor está construido con Express.js y expone un único endpoint que maneja el proceso completo de compilación.

Endpoint: POST /api/compilar

Parámetros del cuerpo (JSON):

- codigoWison: string con la definición de la gramática en lenguaje Wison.
- textoPrueba: string con la cadena de entrada a analizar.

Flujo de procesamiento:

- Valida que ambos campos estén presentes (400 si faltan).
- Invoca parser.parse(codigoWison) para obtener el AST.
- Instancia GestorGramatica con el AST. Si hay errores semánticos, responde 400 con el detalle.
- Calcula First, Follow y la TAS. Si aparecen errores (colisiones), responde 400.
- Instancia Lexer con los terminales de la gramática y genera los tokens del textoPrueba.
- Ejecuta analizarCadenaConPila con los tokens obtenidos.
- Serializa y devuelve: { exito, errores, firsts, follows, tablaTAS, arbol, tokens }.

Manejo global de errores:

Si el parser Jison o el Lexer dinámico lanzan una excepción, el bloque catch la captura y responde { exito: false, errores: [mensaje] } con código 200 para que el frontend pueda procesarlo.

Configuración del servidor:

- Puerto: 3000.
- CORS habilitado globalmente (cors()) para permitir peticiones desde el frontend Angular en desarrollo.
- Límite de cuerpo JSON: valor por defecto de Express (100kb), suficiente para gramáticas típicas.

Interfaz de Usuario – Angular

Está compuesto por tres archivos principales:

api.ts – Servicio de Comunicación

Encapsula la llamada HTTP al backend. El método enviarParaCompilar recibe la gramática y el texto de prueba, construye el objeto de petición y retorna un Observable de Angular para manejo.

app.ts – Componente Principal

Maneja el estado de la aplicación:

- gramatica: string – contenido del textarea de la gramática Wison.
- textoAEvaluar: string – cadena de prueba ingresada por el usuario.
- resultados: any – respuesta JSON del backend (First, Follow, TAS, árbol, errores).
- cargando: boolean – controla el estado del botón durante la petición HTTP.
- columnasTAS: string[] – lista de terminales únicos para renderizar las columnas de la TAS.

El método compilar() valida que ambos campos tengan contenido, activa el indicador de carga, llama al servicio API y asigna la respuesta a resultados. El método obtenerTerminales() extrae todos los terminales únicos presentes en la TAS para construir las columnas de la tabla.

app.html – Plantilla

La interfaz está dividida en tres secciones principales:

- Panel de entrada: textarea para la gramática Wison, input para la cadena de prueba y botón 'Compilar y Generar Tablas'. El botón se deshabilita mientras cargando sea true.
- Panel de errores: visible solo cuando resultados.errores tiene elementos. Lista todos los mensajes de error devueltos por el backend.
- Panel de resultados visible solo si resultados.exito es true: muestra las tablas de First y Follow en paralelo, la Tabla Predictiva TAS con scroll horizontal, y el árbol de derivación visual.

El árbol de derivación se genera mediante el ng recursivo (nodoArbol) que itera sobre los hijos de cada nodo. Los nodos hoja (sin hijos) reciben una clase CSS especial (es-hoja) con color verde para distinguirlos visualmente de los nodos intermedios.

app.css – Estilos

Implementa el árbol de derivación usando CSS puro con ::before y ::after, sin librerías externas. Las líneas del árbol se dibujan mediante bordes en los contenedores de los hijos, logrando el efecto visual de ramas que conectan nodos padre con hijos.
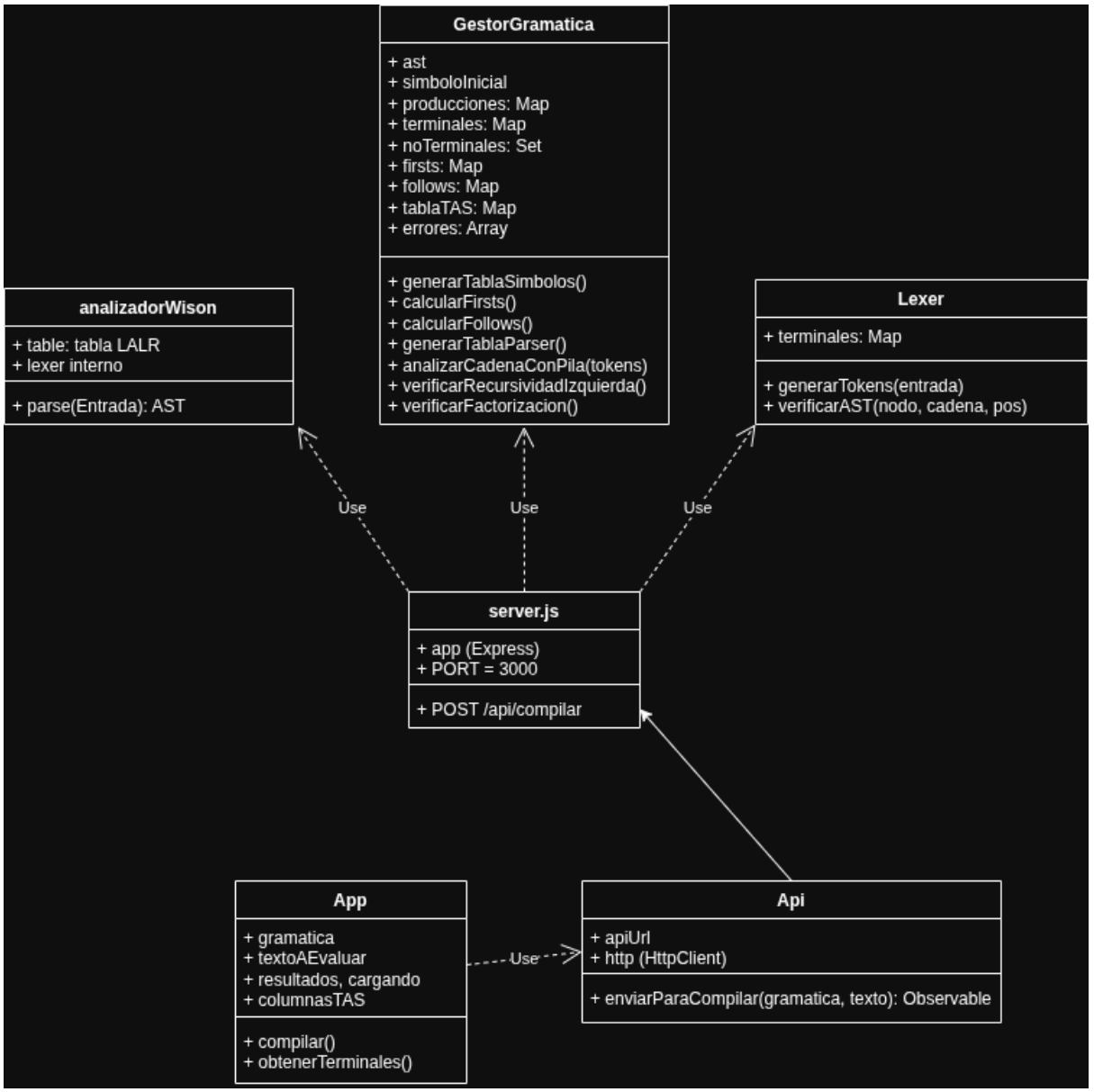
Diagrama de Clases